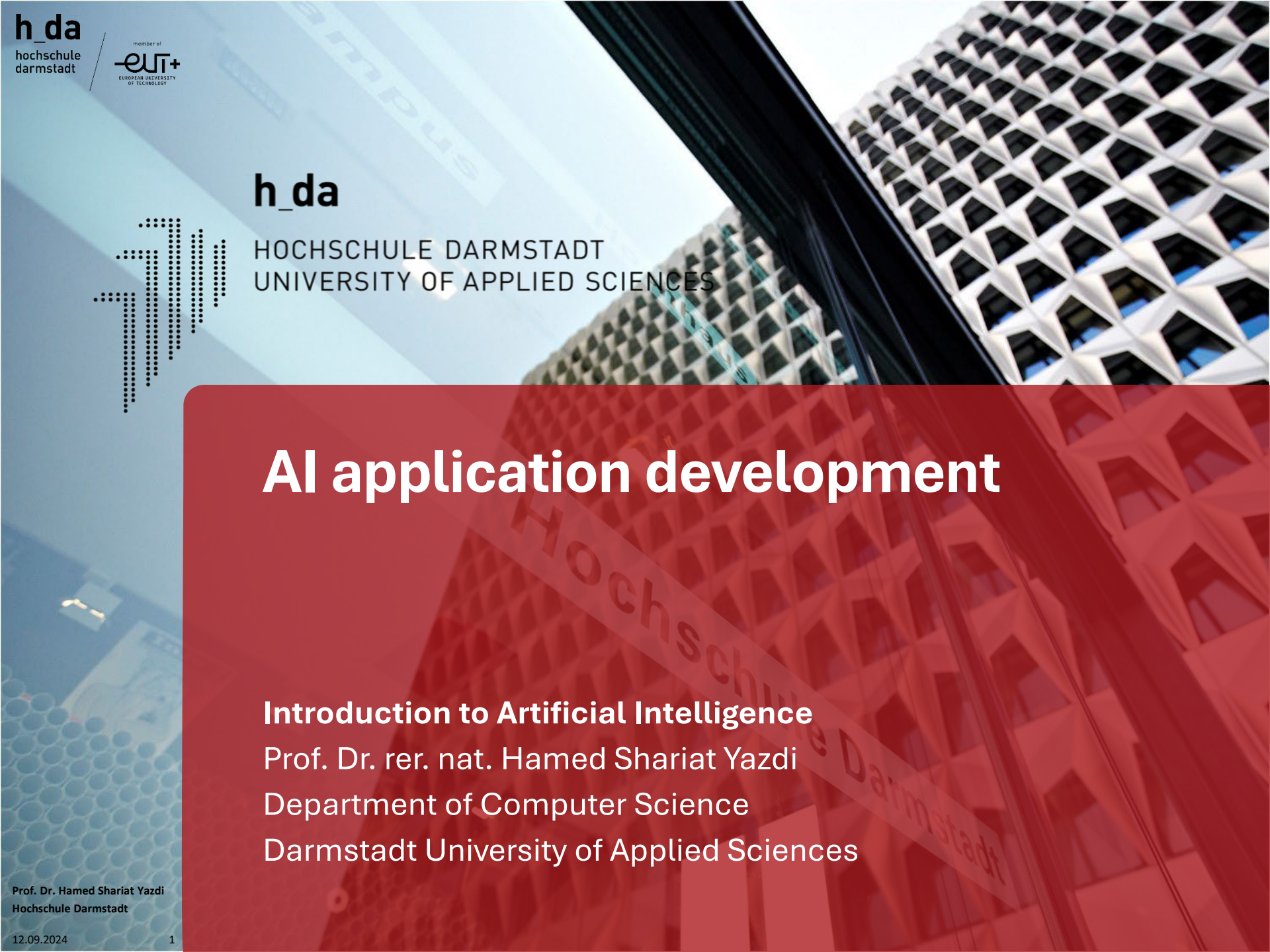


AI application development

Introduction to Artificial Intelligence

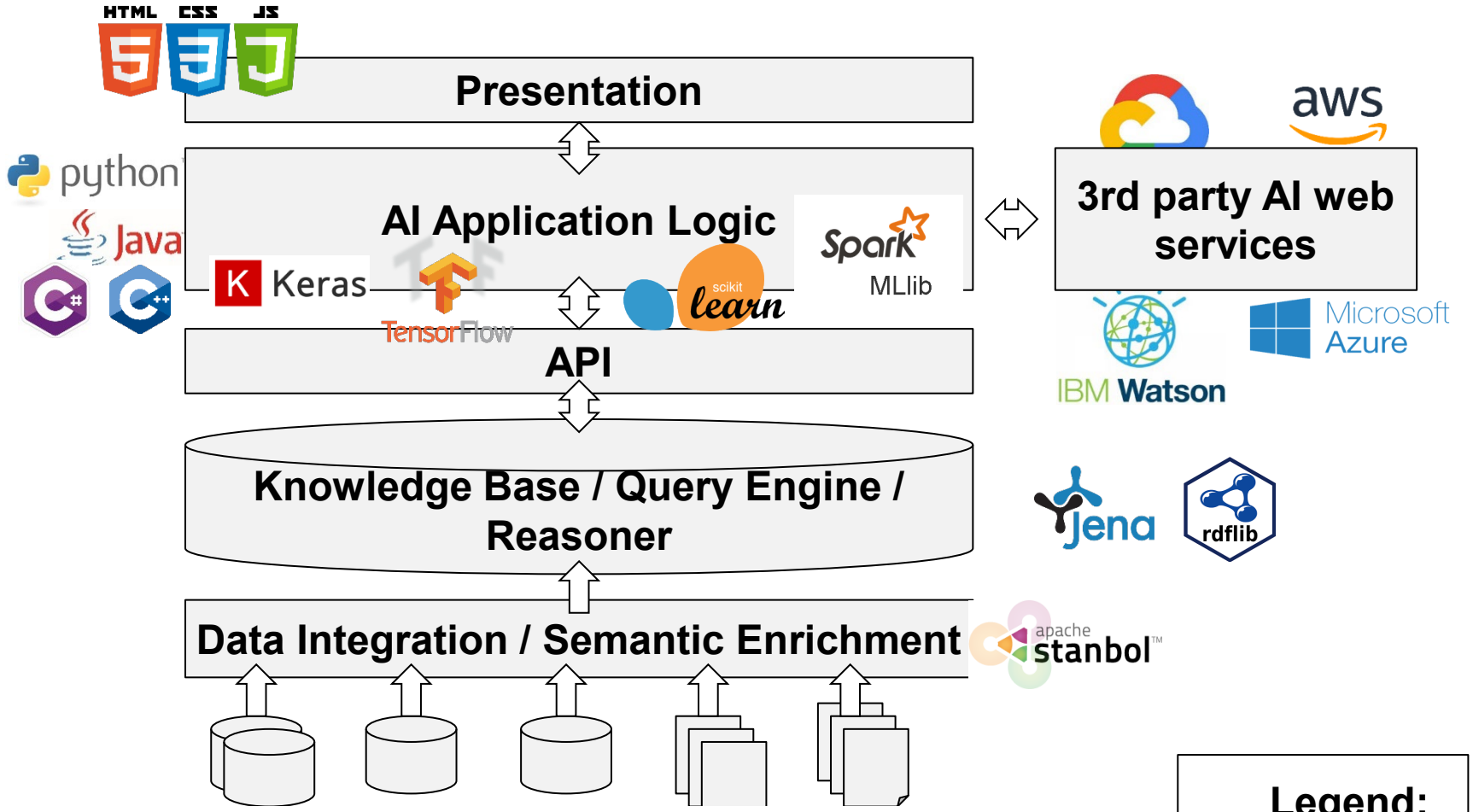
Prof. Dr. rer. nat. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt University of Applied Sciences

Agenda

- AI application reference architecture
- A simplified example
- Quick check

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

AI Application Reference Architecture



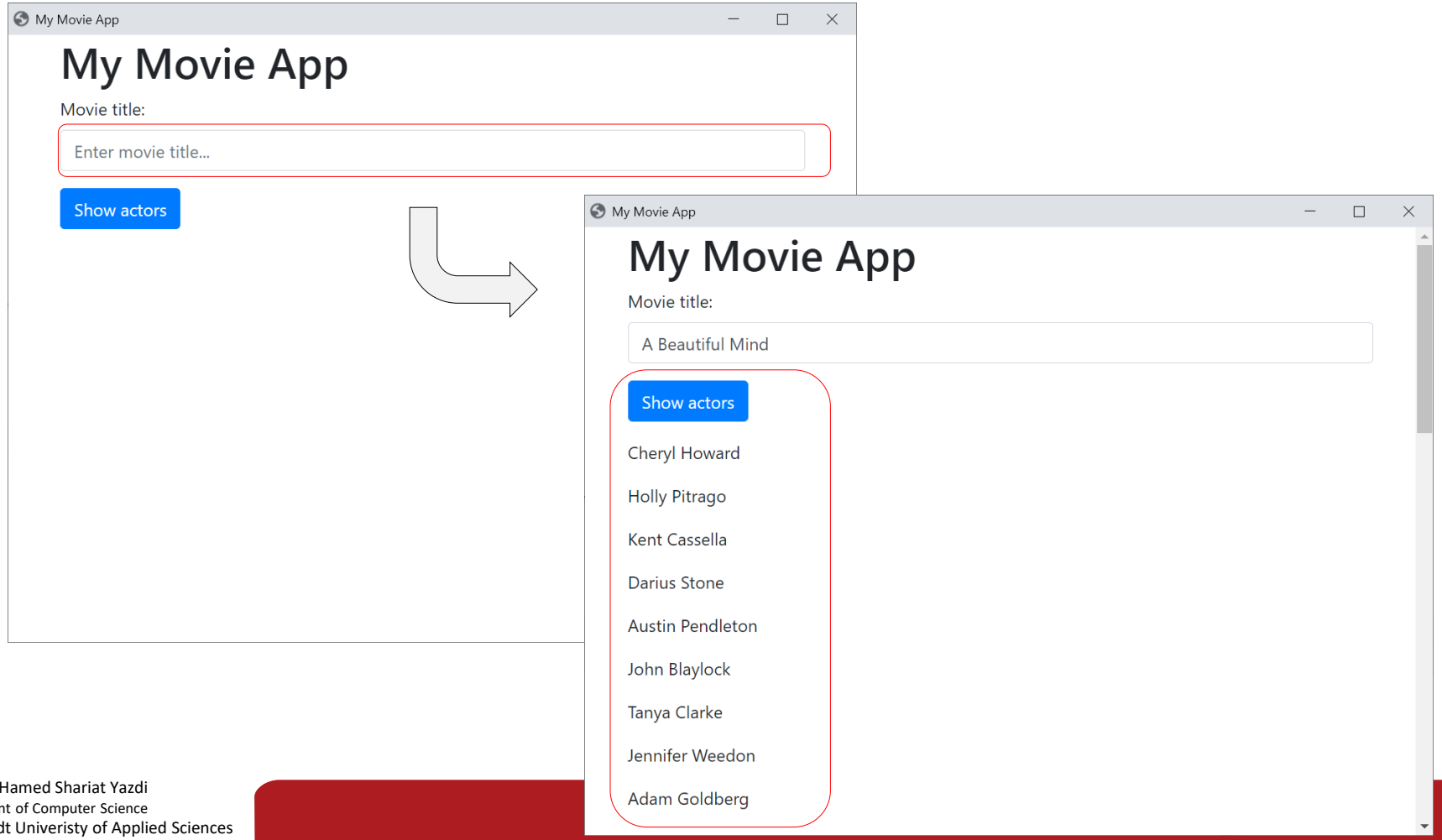
Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Scie

Agenda

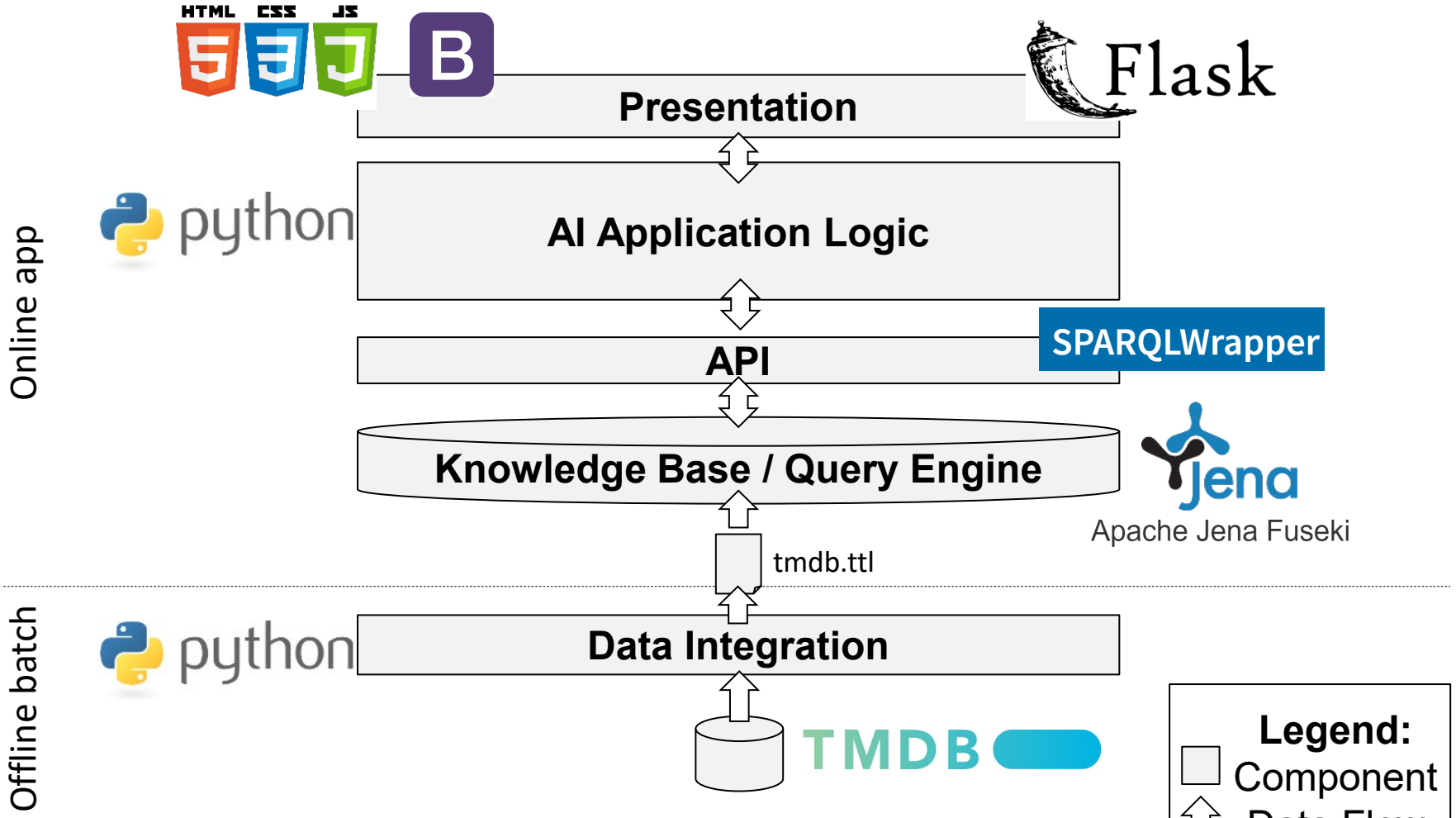
- AI application reference architecture
- A simplified example
- Quick check

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Simplified application example: My Movie App



Software Architecture



Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Flask web development framework



Project Links

Donate
PyPI Releases
Source Code
Issue Tracker
Chat

Contents

Welcome to Flask
User's Guide
API Reference
Additional Notes

Quick search

Go

Flask

Welcome to Flask's documentation. Get started with Installation and then get an overview with the Quickstart. There is also a more detailed Tutorial that shows how to create a small but complete application with Flask. Common patterns are described in the Patterns for Flask section. The rest of the docs describe each component of Flask in detail, with a full reference in the API section.

Flask depends on the Werkzeug WSGI toolkit, the Jinja template engine, and the Click CLI toolkit. Be sure to check their documentation as well as Flask's when looking for information.

User's Guide

Flask provides configuration and conventions, with sensible defaults, to get started. This section of the documentation explains the different parts of the Flask framework and how they can be used, customized, and extended. Beyond Flask itself, look for community-maintained extensions to add even more functionality.

- Installation
  - Python Version
  - Dependencies
  - Virtual environments
  - Install Flask
- Quickstart
  - A Minimal Application
  - Debug Mode
  - HTML Escaping
  - Routing
  - Static Files

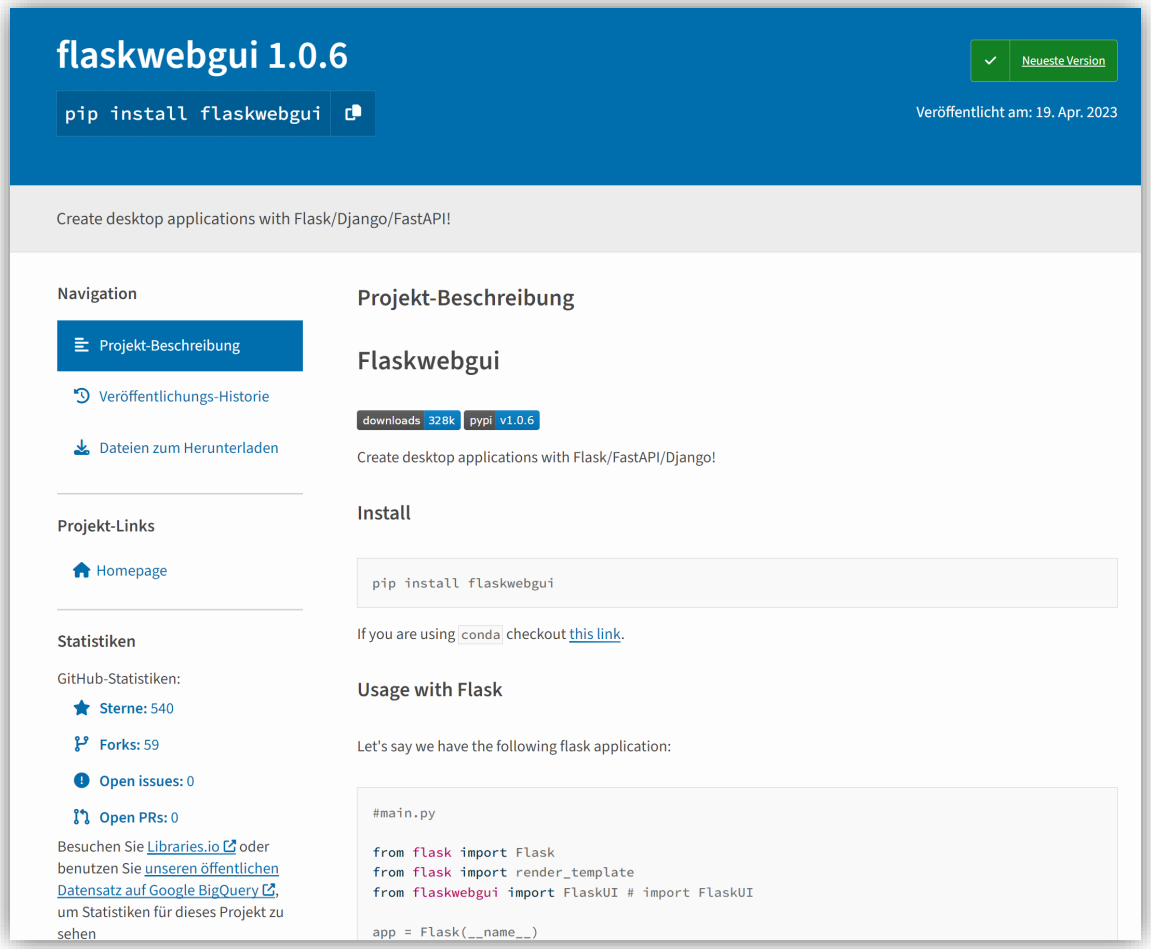
v: 2.3.x

Flask tutorial: https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

https://flask.palletsprojects.com

Flask Web GUI
Convenience wrapper for Flask



https://pypi.org/project/flaskwebgui

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Jinja template engine



Jinja is a modern and designer-friendly templating language for Python, modelled after Django's templates. It is fast, widely used and secure with the optional sandboxed template execution environment:

```
<title>{% block title %}{% endblock %}</title>
<ul>
{% for user in users %}
  <li><a href="{{ user.url }}">{{ user.username }}</a></li>
{% endfor %}
</ul>
```

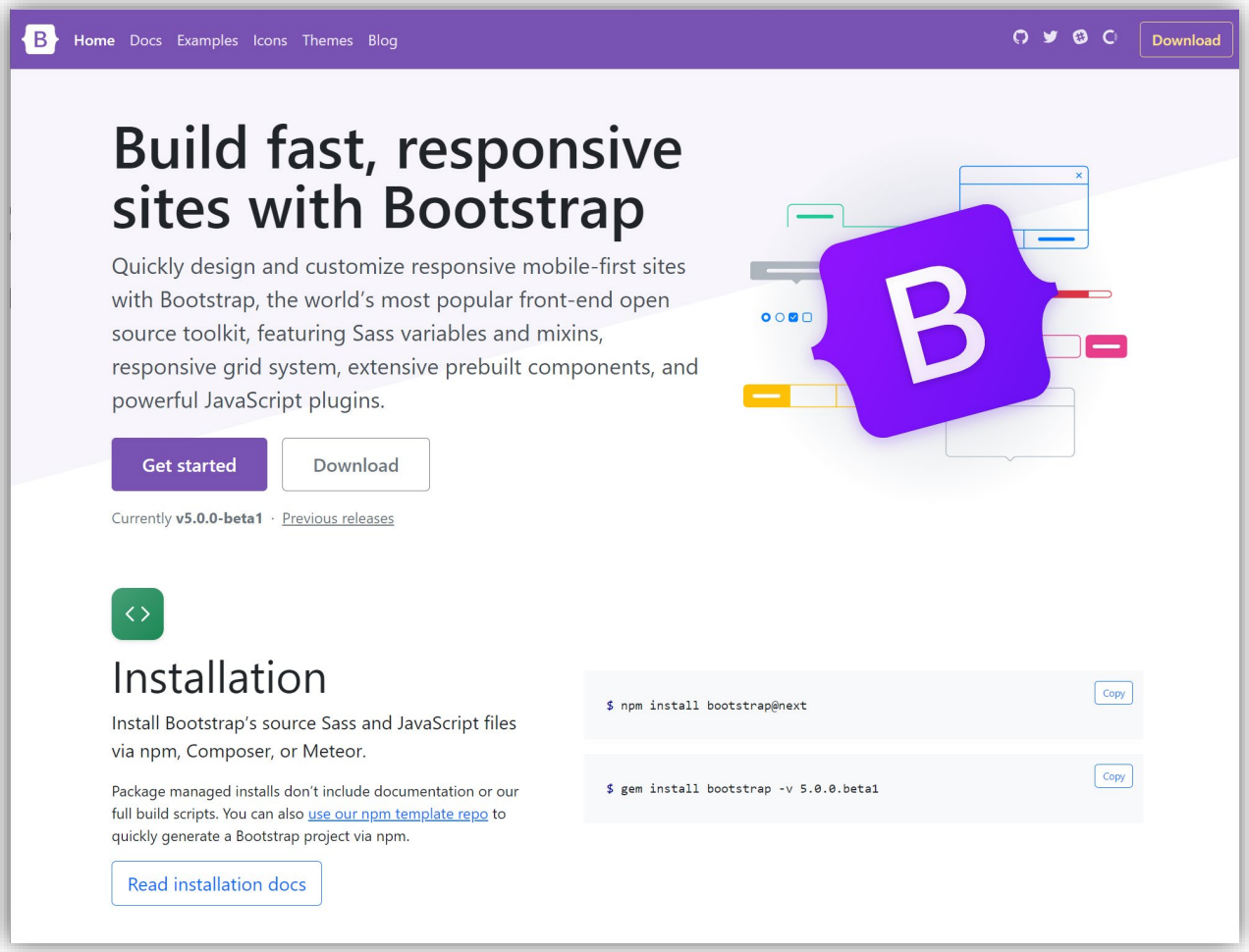
Features:

- sandboxed execution
- powerful automatic HTML escaping system for XSS prevention
- template inheritance
- compiles down to the optimal python code just in time
- optional ahead-of-time template compilation
- easy to debug. Line numbers of exceptions directly point to the correct line in the template.
- configurable syntax

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

https://jinja.palletsprojects.com

Bootstrap Web toolkit



https://getbootstrap.com

Flask

flaskwebgui

app.py
Web application logic

```
pip install flaskwebgui
```

```
from flask import Flask, render_template, request
from flaskwebgui import FlaskUI

app = Flask(__name__)
ui = FlaskUI(app)

@app.route("/", methods=('GET', 'POST'))
def index():
   if request.method == 'POST':
      title = request.form['title']
      return render_template('index.html', actors=query(title))
   else:
      return render_template('index.html', actors=[])

ui.run()
```

Instantiation of Flask app and FlaskUI (from flaskwebgui)

Definition of single page “index” with route and HTTP methods

Extract input (title) from page

Execute application logic and return output (actors) to page

Open page in new browser window on http://localhost:5000

Flask tutorial:
https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

SPARQLWrapper

app.py
SPARQL query logic

```
pip install SPARQLWrapper
```

```
from string import Template
from SPARQLWrapper import SPARQLWrapper, JSON
sparql = SPARQLWrapper("http://localhost:3030/TMDB/sparql")

actor_query = """
PREFIX : <https://www.themoviedb.org/kaggle-export/
SELECT ?actor
WHERE{?m a :Movie ; :title "$title" ;  :cast/:name ?actor .}"""

def query(title):
    query_string = Template(actor_query).substitute(title=title)
    sparql.setQuery(query_string)
    sparql.setReturnFormat(JSON)
    results_dict = sparql.query().convert()
    results = [row['actor']['value'] for row in results_dict['results']['bindings']]
    return results
```

Connection to running Fuseki server with tmdb.ttl loaded

SPARQL query with template parameter

Template substitution

Query preparation and execution

Extract results

index.html
Homepage definition






```
{% extends 'base.html' %}

{% block content %}
  <h1>{% block title %} My Movie App {% endblock %}</h1>

  <form method="post">
    <div class="form-group">
      <label>Movie title:</label>
      <input type="text" name="title"
          placeholder="Enter movie title..." class="form-control"
          value="{{ request.form['title'] }}"/>
    </div>
    <div class="form-group">
      <button class="btn btn-primary">Show actors</button>
    </div>
  </form>

  {% for actor in actors %}
    <p>{{ actor }}</p>
  {% endfor %}

{% endblock %}
```

Reference to base HTML specification including CSS

Jinja templates surrounded by {% … %}

HTML tags as usual

Value binding (used in app.py)

Displaying values computed by app.py

Flask tutorial: https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3

base.html
CSS and JavaScript specification

Standard HTML

Link Bootstrap CSS

Provide headers / footers here to be used in all pages (if needed)

JavaScript sources

```
<!doctype html>
<html lang="en">
 <head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no"

  <!-- Bootstrap CSS -->
  <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUO

  <title>{% block title %} {% endblock %}</title>
 </head>
 <body>
  <div class="container">
    {% block content %} {% endblock %}
  </div>

  <!-- Optional JavaScript -->
  <!-- jQuery first, then Popper.js, then Bootstrap JS -->
  <script src="https://code.jquery.com/jquery-3.3.1.slim.min.js" integrity="sha384-q8i/X+965DzO0rT7abK41JStQIAqVgRVzpbzo5smXKp4YfRvH+8abtTE1Pi6jizo" crossorigin="anonymo
  <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.14.7/umd/popper.min.js" integrity="sha384-UO2eT0CpHqdSJQ6hJty5KVphtPhzWj9WO1clHTMGa3JDZwrnQq4sF86dI
  <script src="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/js/bootstrap.min.js" integrity="sha384-JjSmVgyd0p3pXB1rRibZUAYoIIy6OrQ6VrjIEaFf/nJGzIxFDsf4x0xIM+B07jRM"
 </body>
</html>
```

Flask tutorial: https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3

Agenda

- AI application reference architecture
- A simplified example
- Quick check

Quick check
AI application development

1. Explain the AI reference architecture